MODEL CONTEXT PROTOCOL · DEEP DIVE

Module 2

Everyday MCP I — Using MCP in Claude

Claude Desktop config · Custom Connectors · Claude Code CLI · scopes · filesystem & fetch · hands-on lab

1 HOUR | HANDS-ON LAB | INSTRUCTOR-LED

BestITCourses

What This Module Covers

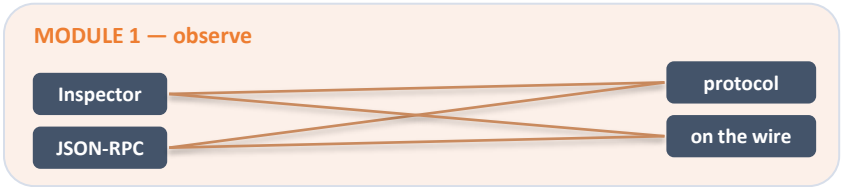
1. **Recap & Framing** — From Module 1 vocabulary to using MCP in Claude
2. **Claude Desktop — Local Servers** — claude_desktop_config.json · the mcpServers key
3. **MCP Indicator & Approval Model** — per-action consent · tools visible in the input box
4. **Remote Servers — Custom Connectors** — remote URL + OAuth/API-key · per-tool permissions
5. **Claude Code CLI** — claude mcp add · stdio & http · /mcp · scopes
6. **Scopes — local / project / user** — ~/.claude.json & team-shared .mcp.json
7. **Hands-On Lab** — filesystem + fetch servers · read a file · fetch a URL

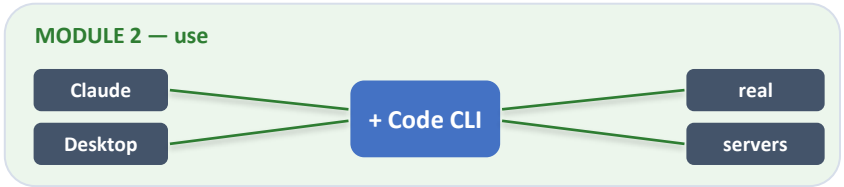
From Observing the Protocol to Using It

In Module 1 you **observed the protocol on the wire** with the MCP Inspector. In Module 2 you become the operator — wiring up real MCP servers in the AI host you actually use: Claude.

- **The host:** Claude ships in two host forms — Claude Desktop and Claude Code — both configurable with MCP servers.
  - The transports: **stdio & HTTP** — local servers launched on your machine over stdio; remote servers reached over the internet via Streamable HTTP.
  - Linux participants: skip Claude Desktop steps; use Claude Code, which is fully cross-platform.

- **The goal today: fluent use** of MCP as an end user and operator. Building servers starts in Module 4.
  - By the end: filesystem + fetch servers wired into Claude Desktop and Claude Code, driving real tool calls with per-action approval.





Two Claude Host Forms — Desktop & Code

How each host adds servers

- Claude Desktop (macOS / Windows) — edit claude_desktop_config.json, or one-click Extensions / Connectors
- Claude Code CLI (macOS / Linux / Windows) — claude mcp add … and the /mcp slash command
- Both use the same mcpServers JSON shape; both support stdio (local) and Streamable HTTP (remote)
- Linux note: Claude Desktop is macOS/Windows only — use Claude Code on Linux

Where configuration lives

Host	Config store
Claude Desktop	claude_desktop_config.json (per-OS path)
Claude Code (local / user)	~/.claude.json (a single dotfile)
Claude Code (project)	.mcp.json in the project root — team-shared, checked into git

Key rule: both hosts read configuration only at startup — edits require a full restart of Claude Desktop, or a fresh Claude Code session.

Local Servers in Claude Desktop

Concept	What it means in Claude Desktop
The host	Claude Desktop launches each configured MCP server as a subprocess and speaks to it over stdio
The config	claude_desktop_config.json holds an mcpServers object — one entry per server (command + args + env)
Approval	Every tool call shows a prompt — you approve or deny each action, every time

One host → many MCP servers → each one a stdio subprocess managed by Claude Desktop.

Example (filesystem server):

configure the @modelcontextprotocol/server-filesystem package with a list of allowed directories; Claude Desktop launches it via npx and the tools appear in the input box.

Locating claude_desktop_config.json

macOS

- Open from Claude: **Claude menu → Settings… → Developer → Open App Config File**
- Path: **~/Library/Application Support/Claude/claude_desktop_config.json**
- Older builds label the button Edit Config — same file.
- Top-level JSON key holding all servers is mcpServers (case-sensitive).

Windows

- Open from Claude: **Claude menu → Settings… → Developer → Open App Config File**
- Path: **%APPDATA%\Claude\claude_desktop_config.json**
- Expands to C:\Users\<You>\AppData\Roaming\Claude\claude_desktop_config.json — the exact path the dialog shows.
- **Note:** JSON strings need escaped backslashes — use C:\\Users\\<You>\\... (double-backslash) inside the file.

- **Note:** the file is read only at startup — a full quit & relaunch is required.

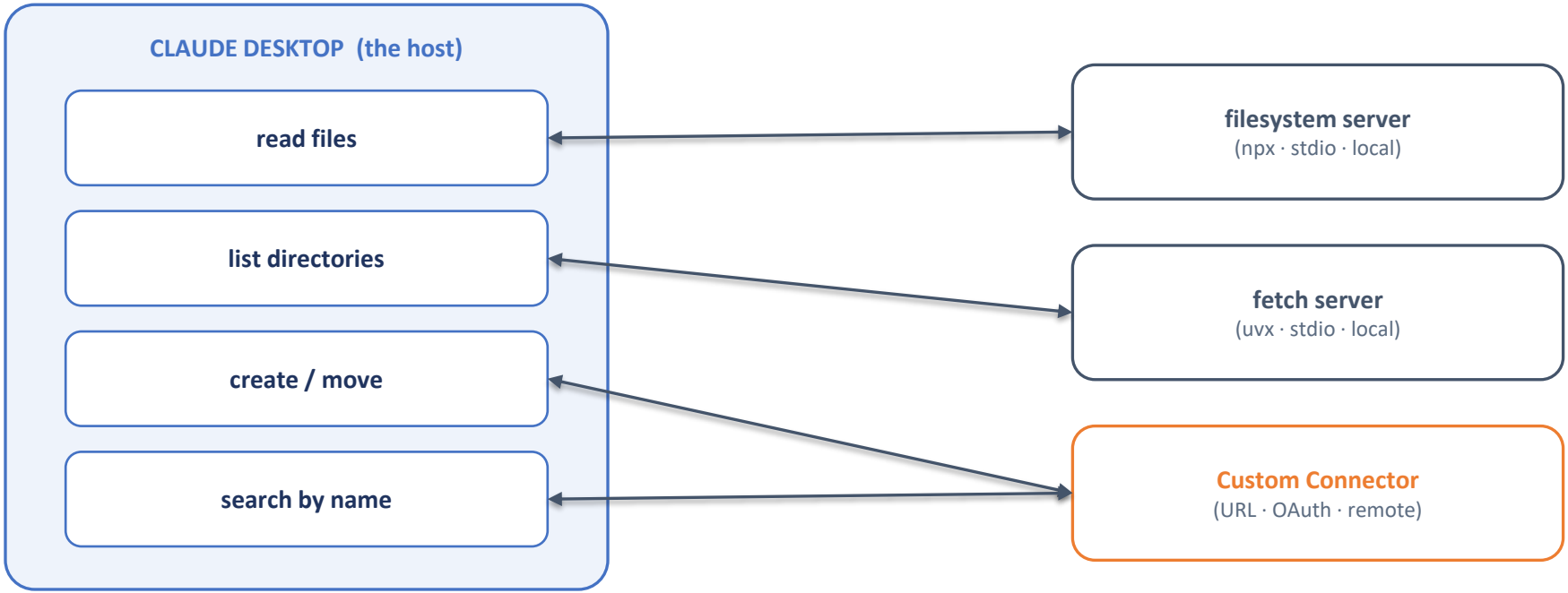
mcpServers — the JSON Configuration Shape

```
// macOS
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/you/Desktop",
        "/Users/you/Downloads"
      ]
    }
  }
  }
}
```

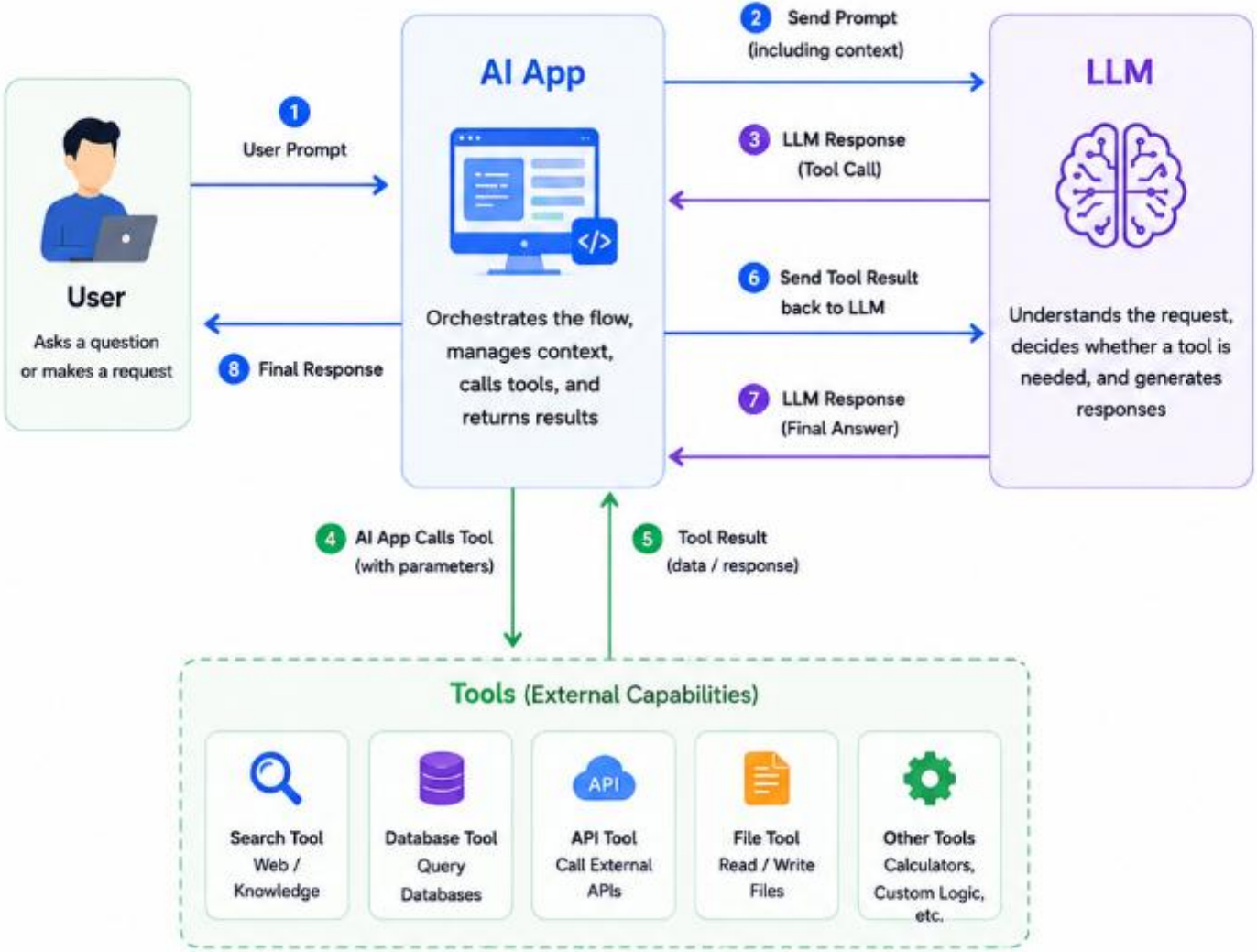
```
// Windows
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-
filesystem",
        "C:\\Users\\you\\Desktop"
      ]
    }
  }
  }
}
```

- **filesystem** — friendly server name that appears in Claude Desktop (yours to choose).
- **command + args** — npx -y auto-installs the package; remaining args are the directories the server may access.
- **Identity exchange** — clientInfo / serverInfo for debugging; then notifications/initialized

What the Filesystem Server Provides



Every tool call from any of these servers requires your explicit approval — Claude shows the exact action before running it.


User
Asks a question or makes a request
1 User Prompt
8 Final Response
AI App
Orchestrates the flow, manages context, calls tools, and returns results
2 Send Prompt (including context)
3 LLM Response (Tool Call)
6 Send Tool Result back to LLM
7 LLM Response (Final Answer)
LLM
Understands the request, decides whether a tool is needed, and generates responses
4 AI App Calls Tool (with parameters)
5 Tool Result (data / response)
Tools (External Capabilities)
Search Tool Web / Knowledge
Database Tool Query Databases
API
API Tool Call External APIs
File Tool Read / Write Files
Other Tools Calculators, Custom Logic, etc.

One-Click Options: Extensions & Remote Connectors

Beyond hand-editing JSON, Claude Desktop offers two managed paths to add servers — useful for non-developers and remote services:

Option	What it is	When to use
Desktop Extensions	Packaged MCP servers installable with one click — no JSON editing	For widely-used local servers and non-technical users
Remote Connectors	Connect to remote MCP servers over HTTP with OAuth (some require a paid plan)	For SaaS / hosted services like Sentry, GitHub, etc.

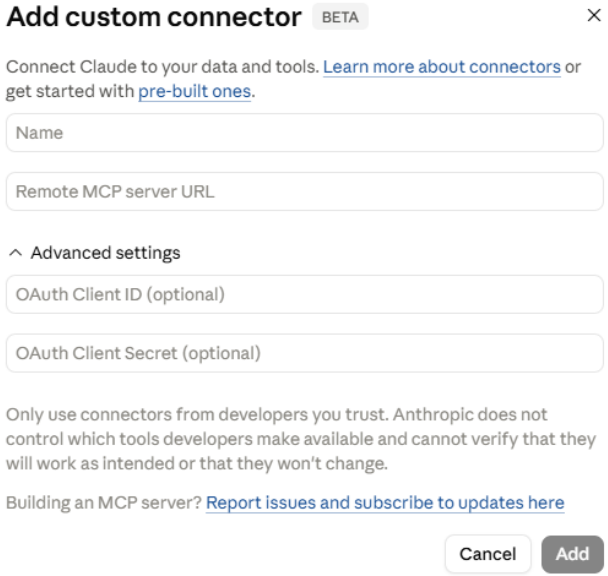
*Extensions live in: profile icon → Settings → Extensions (These are saved in **%APPDATA%\Claude**, in the Extensions storage / app settings and not in any JSON config.*

Connectors live in: Left Menu → Customize → Connectors (These are connected to Claude Account and not saved locally.

Heads-up — trust matters: only connect to remote MCP servers from sources you trust; review the OAuth scopes the server requests before approving.

Remote Servers

For Custom Connectors - Open Customize settings → + Icon → Add custom connector



Adding a Local (stdio) Server in Claude code

```
    # Template — options BEFORE name, --
                  separator
claude mcp add \
  [--transport stdio] \
  [--env KEY=value] \
  [--scope local|project|user] \
  <name> -- <command> [args...]

claude mcp list

claude mcp remove <name>
```

- **Options before name** — all flags must come before <name>, then -- separates Claude flags from server args.
- **--transport stdio** — default; safe to omit when launching local subprocesses.
- **--env KEY=value** — pass env vars (API keys, paths). Repeat for multiple.
- **--scope** — local (default, this project), project (.mcp.json), user (all your projects).

```
            # Example — filesystem server (user scope)
claude mcp add --scope user filesystem \
  -- npx -y \
  @modelcontextprotocol/server-filesystem \
  "$HOME/mcp-lab-sandbox"

# Windows (PowerShell) — forward slashes!
Powershell
$s = "$HOME\mcp-lab-sandbox" -replace '\\','/'
claude mcp add --scope user filesystem -- npx
"@modelcontextprotocol/server-filesystem" "$s"

-------------------------------------------------------------

# Fetch server (Python, via uvx)
claude mcp add --scope user fetch -- uvx mcp-server-fetch
```

Other MCP Commands:

- claude mcp list
- claude mcp remove filesystem

Updates: C:\Users\<user>\.claude.json

Scopes — local / project / user

Every claude mcp add writes to one of three scopes, which determines **where** the server definition is stored and ***who*** can see it. Default is local. Project scope is the only one that writes a repo file.

Scope	What it does	Stored in
local (default)	This project only, private to you	~/.claude.json (under **projects[].mcpServers**)
project	Team-shared — committed with the repo	.mcp.json in the project root (commit me)
user	All your projects, private to you	~/.claude.json (top-level mcpServers — global)

- **~/.claude.json** on macOS/Linux; %USERPROFILE%\.claude.json on Windows — a single dotfile, not a folder.
- **.mcp.json** uses the same mcpServers JSON shape as Claude Desktop, and supports stdio & http entries side by side.

Adding a Remote (HTTP / SSE) Server in Claude code

```
# HTTP (recommended for remote)

claude mcp add --transport http <name> <url>

# With a bearer token
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ --header "Authorization: Bearer
YOUR_PAT" –scope project

# SSE (deprecated — prefer HTTP)
claude mcp add --transport sse <name> <url> --header "X-API-Key: …"
```

Use **`--transport http`** for remote MCP servers. SSE is supported but deprecated. Pass auth as **`--header`** values, or use add-json for an exact JSON definition (handy for SaaS connectors).

Adding from JSON (any transport)

```
                                        # Add from JSON (any transport)

claude mcp add-json local-fetch \
  '{
     "type":"stdio",
     "command":"uvx",
     "args":["mcp-server-fetch"]
   }'

claude mcp add-json weather \
  '{
      "type":"http",
      "url":"https://api.example.com/mcp",
      "headers":{"Authorization":"Bearer t"}
   }'

# Note: store URLs in trusted sources only
```

Inspect, Manage & the /mcp Slash Command

```
# Inspect & manage
claude mcp list
claude mcp get filesystem
claude mcp remove filesystem
claude mcp add-from-claude-desktop
claude mcp serve   # expose Claude Code as a server
```

- **claude mcp list** — shows all configured servers with **connection status and tool counts**
- **claude mcp get / remove** — inspect a single server (resolved command, scope) or remove one entirely
- **add-from-claude-desktop** — import Claude Desktop servers into Claude Code (macOS / WSL)

The /mcp slash command — used inside an interactive Claude Code session:

```
# Inside Claude Code (interactive)
> /mcp
┌──────────────────────────────────┐
│ filesystem  ✓ connected  5       │
│ fetch       ✓ connected  1       │
│ github      ! 401 (re-auth)      │
```

/mcp shows status & tool counts and drives OAuth for remote servers that returned 401/403; it does NOT add servers — use claude mcp add for that.

Hands-On Lab — Drive filesystem + fetch from Claude

Goal: configure the filesystem and fetch reference servers in both Claude Desktop and Claude Code, then have Claude read a local file and fetch a URL — observing the **per-action approval prompt** end to end. (~25–30 min)

- **Prereqs:** Node.js 18+ (npx), uv (uvx mcp-server-fetch), Claude Desktop and/or Claude Code installed
- **Recreate the sandbox (Linux/macOS):**

```
mkdir -p "$HOME/mcp-lab-sandbox" && printf 'Hello from MCP.\n' > "$HOME/mcp-lab-sandbox/hello.txt"
```

- Part A (macOS / Windows) — Claude Desktop: edit claude_desktop_config.json with both servers, full restart, verify MCP indicator
- Part B (Linux / macOS / Windows) — Claude Code: claude mcp add filesystem & fetch (user scope); claude mcp list; /mcp in-session
- Drive both from Claude: **read hello.txt** from the sandbox via the filesystem server; fetch modelcontextprotocol.io/llms.txt via the fetch server; approve each call.

Claude Desktop — Wire Up filesystem + fetch

Action Checklist

Five steps to add both reference servers, restart, and run a tool with approval.

· Step A1: Claude menu → Settings… → Developer → Open App Config File (older builds: Edit Config).

· Step A2: Replace the file with the OS-specific JSON shown (substitute your real absolute sandbox path).

· Step A3: Fully quit Claude Desktop (not just close the window) and relaunch — config is read only at startup.

· Step A4-A5: Confirm the MCP indicator appears, then ask Claude to read hello.txt and fetch a URL — approve each call.

claude_desktop_config.json

```
// macOS
{ "mcpServers": {
  "filesystem": {

    "command": "npx",
    "args": ["-y",
      "@modelcontextprotocol/server-filesystem",

      "/Users/you/mcp-lab-sandbox"]
  },
  "fetch": {
    "command": "uvx",
    "args": ["mcp-server-fetch"]

  } } }

// Windows — paths need C:\\Users\\you\\…
// (double-backslash inside JSON)
// then: Quit Claude, Relaunch, check indicator.
```

Claude Code — claude mcp add & the /mcp Panel

Step B1-B3: Add servers + verify

Run these in any terminal (Linux / macOS / Windows-PowerShell):

· **Filesystem (user scope):** claude mcp add --scope user filesystem -- npx -y @modelcontextprotocol/server-filesystem "$HOME/mcp-lab-sandbox"

· **Fetch (user scope):** claude mcp add --scope user fetch -- uvx mcp-server-fetch

· **Verify:** claude mcp list → both servers should appear; claude mcp get filesystem shows the resolved command.

· **Windows tip:** backslashes get stripped by the npx shim — pass a forward-slash path ($sandbox = $HOME\mcp-lab-sandbox -replace '\\','/').

· d:\>: start claude, then ask: Read hello.txt from my mcp-lab-sandbox; Fetch modelcontextprotocol.io/llms.txt and summarize.

Step B4-B5: /mcp panel + project scope (.mcp.json)

Inside the Claude Code session, run **/mcp** to see status and tool counts; then prove project scope writes a repo file:

$ mkdir -p ~/mcp-m2-project && cd ~/mcp-m2-project

$ claude mcp add --scope project fetch -- uvx mcp-server-fetch && cat .mcp.json

· /mcp: shows status (connected / pending / failed), tool counts, and drives OAuth for remote servers that returned 401/403.

· .mcp.json: opens a team-shared file with an mcpServers object — this is what you would commit to git for collaborators.

· Cleanup (optional): claude mcp remove filesystem; claude mcp remove fetch.

Knowledge Check: Test Your Understanding

Q1: Config Key & Restart

What top-level JSON key holds servers in claude_desktop_config.json, and why must you fully quit/restart Claude Desktop after editing it?

· Answer: mcpServers. Claude Desktop reads the config only at startup, so changes take effect only after a complete quit-and-relaunch (closing the window is not enough).

Q2: Path Rules

Name two Claude Desktop path rules that prevent the most common load failures (one general, one Windows-specific).

· Answer: General — every path must be absolute (no relative, no ~). Windows — backslashes in JSON strings must be escaped/doubled (C:\\Users\\...).

Q3: CLI Syntax

In claude mcp add, where must options go relative to the server name, and what is the purpose of the -- separator?

· Answer: All options must come before the server name; -- separates Claude's own options from the command/args handed to the MCP server (prevents flag collisions).

Q4: Scopes & /mcp

Which --scope writes .mcp.json in the project root, what is the default scope, and what does the in-session /mcp command do?

· Answer: --scope project writes .mcp.json (team-shared). Default is local. /mcp shows status / tool counts and drives OAuth for remote servers — it does NOT add servers.

Summary & What's Next

- Claude Desktop loads local servers from **claude_desktop_config.json under mcpServers** — absolute paths, escaped Windows backslashes, and a full restart are mandatory.
- **MCP indicator** (slider, bottom-right) confirms load; a **per-action approval prompt** gates **every tool call** — review & approve or deny each one.
- Remote servers connect via **Custom Connectors** (URL + **OAuth / API-key**) with per-tool permissions; only connect to trusted servers.
- Claude Code manages servers with **claude mcp add** (options before name, -- separator); supports **stdio + --transport http|sse**
- Three scopes: **local · project (.mcp.json) · user; in-session /mcp handles status & OAuth.**
- You drove filesystem + fetch from Claude to read a local file and fetch a URL — the **everyday-usage workflow** MCP enables.

Next — Module 3: Everyday MCP II (Copilot, Other Clients & Popular Servers)

VS Code + Copilot agent mode (servers key, not mcpServers!), Cursor / Windsurf / Cline / Goose, the popular reference servers, and the official MCP Registry.